

COLLEGE HACKATHON
MODULE DOCUMENT

MODULE 2

Geo Attendance Tracker

Location-validated attendance for distributed and on-site workforces

Stack: React + Vite • Node.js + Express • MySQL/PostgreSQL • GitHub

Total: 100 Marks | Duration: As per hackathon schedule

Table of Contents

1. Problem Statement & Overview
2. Objectives
3. Required Screens
4. Required APIs
5. Suggested Database Schema
6. Validations
7. Innovation Ideas (Bonus)
8. Module-Specific Rules & Regulations
9. General Hackathon Rules & Regulations
10. Evaluation Criteria (Module-Specific Rubric)
11. Submission Checklist
12. Expected Output

1. Problem Statement & Overview

Build an attendance management system that validates check-in and check-out events against a configured office geofence. The system must capture latitude/longitude during punch events, calculate working hours, detect late arrivals, and provide reports to both employees and admins.

Independent Module Notice

This module is a standalone product. It must be designed and built without dependency on any other hackathon module.

All authentication, user management, and supporting infrastructure must be implemented within this module itself.

2. Objectives

- Capture employee location during check-in and check-out
- Validate that the employee is within the configured office radius
- Prevent multiple check-ins or check-outs without a check-in
- Calculate daily working hours automatically
- Detect and flag late arrivals based on shift start time
- Provide attendance history for employees and dashboards for admins

3. Required Screens

The application must implement the following screens at minimum. Additional screens that improve UX are welcome and may earn innovation marks.

#	Screen	Purpose
1	Login Screen	Email + password login; role-based redirect
2	Employee Dashboard	Today's status, last check-in, this month's summary
3	Check-in / Check-out Page	Big action button; location captured via browser API
4	Attendance History	Calendar view + list with hours per day
5	Admin Dashboard	All employees, today's status, late arrivals, missing punches
6	Office Settings (Admin)	Configure office lat/lng and allowed radius (in meters)
7	Reports Page	Monthly report per employee; export to CSV

4. Required APIs

All APIs must follow REST conventions, return JSON, and use proper HTTP status codes. Authentication via JWT is recommended.

Method	Endpoint	Description
POST	/api/auth/login	Login and receive JWT
POST	/api/attendance/check-in	Punch in with lat/lng
POST	/api/attendance/check-out	Punch out with lat/lng
GET	/api/attendance/today	Today's status for current user
GET	/api/attendance/history?month=YYYY-MM	User's history for a month
GET	/api/admin/attendance/today	All employees' today status
GET	/api/admin/attendance/report?month=YYYY-MM	Org-wide monthly report
GET	/api/settings/office	Get office geofence config
PUT	/api/settings/office	Update office geofence config

5. Suggested Database Schema

Below is a suggested schema. Teams may extend or restructure as long as normalization and referential integrity are preserved. Foreign keys, indexes, and timestamps (created_at / updated_at) are expected.

Table	Columns
users	id, name, email, password_hash, role, shift_start_time, created_at
office_settings	id, latitude, longitude, radius_meters, late_threshold_minutes
attendance	id, user_id, date, check_in_time, check_in_lat, check_in_lng, check_out_time, check_out_lat, check_out_lng, working_hours, is_late, status
attendance_logs	id, user_id, event_type, timestamp, lat, lng, distance_from_office, accepted, reason

6. Validations

All validations below must be enforced on the SERVER. Client-side validation alone is not acceptable.

1. User cannot check in twice on the same day — second attempt rejected with clear error
2. User cannot check out without an existing check-in for the day
3. Distance from office (Haversine formula) must be $\leq$ configured radius; otherwise punch is rejected

4. Lat/Lng must be present and within valid ranges (-90..90, -180..180)
5. Working hours = check_out - check_in, calculated server-side, never accepted from client
6. If check-in time > shift_start_time + late_threshold, mark is_late = true

7. Innovation Ideas (Bonus)

These are optional features that go beyond the required scope. Implementing one or more well can earn innovation marks.

- Map view showing current punch location and office geofence circle
- Auto check-out at end of day for users who forgot to punch out (with flag)
- Mobile-first PWA so it works on phones
- Admin can mark exceptions (work-from-home days, on-duty travel)
- Push notifications / email alerts for late arrivals

8. Module-Specific Rules & Regulations

These rules apply specifically to Module 2 (Geo Attendance Tracker). Violations will result in marks being deducted in the relevant rubric category.

7. Location MUST be obtained from the browser Geolocation API — manual lat/long entry is forbidden.
8. Distance check MUST happen on the server using stored office coordinates. Client-side validation only is not acceptable.
9. Use the Haversine formula (or equivalent geodesic calculation) for distance — flat-earth approximations are not allowed.
10. Working hours MUST be calculated server-side. Never trust the client to send working_hours.
11. Permissions must be requested clearly; deny scenarios must show user-friendly messages.
12. Attendance records, once created, cannot be silently edited. Edits must go to attendance_logs as audit entries.

General Rules & Regulations

All teams must follow the rules below. Violations may result in mark deductions or disqualification.

A. Code of Conduct

13. Each team must consist of the assigned members only. No external help, paid services, or freelancers are permitted.
14. All code must be written during the hackathon window. Pre-written boilerplate beyond the official Vite + Express starter is NOT allowed.
15. Copying repositories, templates, or code from other teams is strictly prohibited and leads to immediate disqualification.
16. AI assistants (Claude, ChatGPT, Copilot, etc.) may be used for guidance and debugging, but the design, architecture, and integration must be the team's own work. Teams must be able to explain every line of code during evaluation.
17. Plagiarism checks will be run on submitted code. Similarity above the threshold will be penalized.

B. Technology Rules

18. Frontend MUST use React.js with Vite. Create-React-App, Next.js, Angular, or Vue are NOT allowed.
19. Backend MUST use Node.js with Express. Other frameworks (NestJS, Fastify, Django, Spring) are not permitted unless explicitly approved.
20. Database MUST be MySQL or PostgreSQL. NoSQL (MongoDB, Firebase) is not allowed for primary data.
21. Use of paid third-party APIs that solve the core problem (e.g., a complete resume-parsing SaaS) is forbidden. Free utility APIs (maps, email, file storage) are allowed and must be disclosed.
22. All secrets (DB passwords, API keys) must be stored in .env files. Hardcoded secrets in committed code will lose marks.

C. Submission Rules

23. Code must be pushed to a public GitHub repository created during the hackathon. Commit history will be reviewed.
24. README.md must include: project description, setup steps, environment variables, default credentials, screenshots, and team member names.
25. Database schema must be submitted as a .sql dump file or migration scripts.
26. API documentation must be provided as Postman collection or OpenAPI/Swagger file.
27. A short demo video (3–5 minutes) walking through the working features must be included in the repo.

28. Late submissions will lose 10 marks per hour of delay, up to a maximum of 30 marks. Beyond 3 hours late, the project will not be evaluated.

D. Evaluation Day Rules

29. Each team gets a fixed presentation slot. Live demo on a working build is mandatory.
30. All team members must be present and must contribute to the presentation.
31. Judges may ask any team member to explain any part of the code or schema.
32. If the application crashes during demo, teams have one chance to recover. Repeated failures result in mark deductions.
33. Decisions of the judging panel are final.

10. Evaluation Criteria — Module 2 Rubric

This module is evaluated using a CUSTOM rubric tailored to its nature. Some modules emphasize UI, others emphasize logic, parsing, or data handling — the weights below reflect what matters most for Geo Attendance Tracker.

Total: 100 marks.

#	Criterion	Marks	What we look for
1	UI / UX (mobile-friendly)	15	Works smoothly on mobile, large action buttons, clear feedback
2	Geolocation Capture	10	Browser API used correctly; permission handling
3	Geofence Validation Logic	20	Haversine done server-side; correct accept/reject behavior
4	Database Design	10	Attendance, audit log, settings tables well-designed
5	Working Hours & Late Detection	10	Server-side calculation; timezone correctness
6	API & Validations	10	Prevents double check-in, missing check-out, invalid coords
7	Reports & Admin Dashboard	10	Useful filters, monthly export, today-at-a-glance view
8	Innovation	10	Map view, PWA, exception handling, notifications
9	Presentation & Demo	5	Clear demo of in-radius and out-of-radius scenarios

Scoring Bands (per criterion)

Score	Meaning
90–100% of criterion marks	Outstanding — exceeds expectations, professional quality
70–89%	Good — meets expectations, minor gaps only
50–69%	Adequate — works, but rough or incomplete in places
30–49%	Weak — partial implementation, multiple issues
0–29%	Poor / missing — broken or absent

How Marks Are Awarded

Each criterion is independently scored against its band, then summed for a total out of 100. Penalties (rule violations, late submission, plagiarism) are subtracted AFTER the rubric total.

Score	Meaning
	Innovation marks are awarded only if the core required features are functional. Innovation cannot compensate for missing core features.

11. Submission Checklist

Submission Checklist

- Public GitHub repository link with full commit history
- README.md with setup, environment variables, and screenshots
- Database schema (.sql dump or migrations folder)
- Sample seed data for demo
- API documentation (Postman collection or Swagger)
- Working deployed link OR clear local setup steps
- Demo video (3–5 minutes)
- List of team members with roles and contributions

12. Expected Output

A fully functional Geo Attendance Tracker application that:

- Runs end-to-end with no critical bugs in the demo path
- Has a clean, responsive UI built in React + Vite
- Uses Node.js + Express on the backend with MySQL/PostgreSQL
- Implements all required screens, APIs, and validations
- Includes proper documentation and a working demo
- Demonstrates the team's understanding of the problem and clean engineering practices

— End of Module Document —